Developers Mania

Working Group

Alfayo Nelson Central Command

Architecture Diagrams

Companion to SRS-ALFAYO-001. Eleven diagrams across four families — context, container, data flow, and sequence — covering the platform’s structure and the workflows that matter most.

Document Control Block

Document Title	Alfayo Nelson Central Command — Architecture Diagrams
Document Reference	ARC-ALFAYO-001
Version	v0.1 (Draft)
Date	26 May 2026
Status	Draft — Pending Peer Review
Classification	Confidential
Product / Project	Alfayo Nelson Central Command Platform
Client	Alfayo Nelson Parliamentary Campaign, Nyali Constituency
Owner	Technical Lead — Developers Mania Working Group
Companion to	SRS-ALFAYO-001 v0.2 (Software Requirements Specification)
Source of Truth	/arch/src/*.mmd — Mermaid source files; this document embeds rendered PNGs

READING NOTE	This document presents diagrams. The diagrams themselves are not the design; they are visualisations of the design captured in SRS-ALFAYO-001. Where a diagram and the SRS disagree, the SRS is authoritative. Diagrams will be updated alongside the SRS as the design evolves; see the source Mermaid files in the project repository for the latest renderings.

1. Introduction

1.1 Purpose

This document presents the architecture of the Alfayo Nelson Central Command platform through eleven diagrams across four families. It supports the design, review, onboarding, and operational handover of the platform. Each diagram is accompanied by commentary describing what it shows, what it deliberately omits, and which SRS requirements it traces to.

1.2 The Four Diagram Families

Architectural diagrams age fast and degrade in usefulness when authors mix levels of abstraction in a single diagram. This document is disciplined about that. Each diagram belongs to exactly one of four families:

Family	What It Shows	When to Use
Context	The platform as a single box. External users and external systems around it.	Kickoff meetings; executive briefings; understanding the system's boundary.
Container	Major deployable parts of the platform. Apps, services, databases, queues.	Developer onboarding; design discussions about what runs where.
Data Flow (DFD)	How data moves between processes and stores. Functional decomposition.	Compliance reviews; data privacy impact assessments; audit walkthroughs.
Sequence	Time-ordered interactions for specific workflows.	Debugging design ambiguity; specifying API behaviour; reviewing failure modes.

1.3 Diagrams in This Document

#	Family	Diagram	Primary SRS Trace
1	Context	System Context	Section 2.1, Section 5.1
2	Container	Container / Component	Section 2.3, Section 5.2
3	DFD	Data Flow — Level 0	Section 0.2.1, Section 2.1
4	DFD	Data Flow — Level 1	Section 3 (all domains)

#	Family	Diagram	Primary SRS Trace
5	ERD	Entity Relationship	Section 3.2–3.15 (all FR domains)
6	Sequence	Login with 2FA	FR-001, FR-002, NFR-013
7	Sequence	Offline Visit Log + Sync	FR-050, FR-100, NFR-002
8	Sequence	Create Meeting + Notify	FR-060, NFR-022 fallback path
9	Sequence	Register Committed Supporter	FR-130 to FR-134, COMP-010
10	Sequence	Election-Day Station Report	FR-120 to FR-122, NFR-021
11	Sequence	Generate Coverage Heatmap	FR-051, NFR-005

1.4 What These Diagrams Deliberately Do Not Cover

Several useful diagram types are intentionally out of scope here, to keep this document focused and prevent it becoming a Technical Specification by accident:

- Cloud deployment topology (VPCs, subnets, load balancers, autoscaling groups) — belongs in the Infrastructure Document produced before staging deployment.
- CI/CD pipeline visualisation — belongs in the DevOps Runbook.
- Monitoring and alerting architecture — belongs in the Operations Manual produced before launch.
- Detailed UI wireframes and screen flows — belong in the UX Specification (separate Phase 2 deliverable).
- Database physical schema (indexes, partitions, materialised views) — belongs in the Database Design Document.

1.5 How to Read the Diagrams

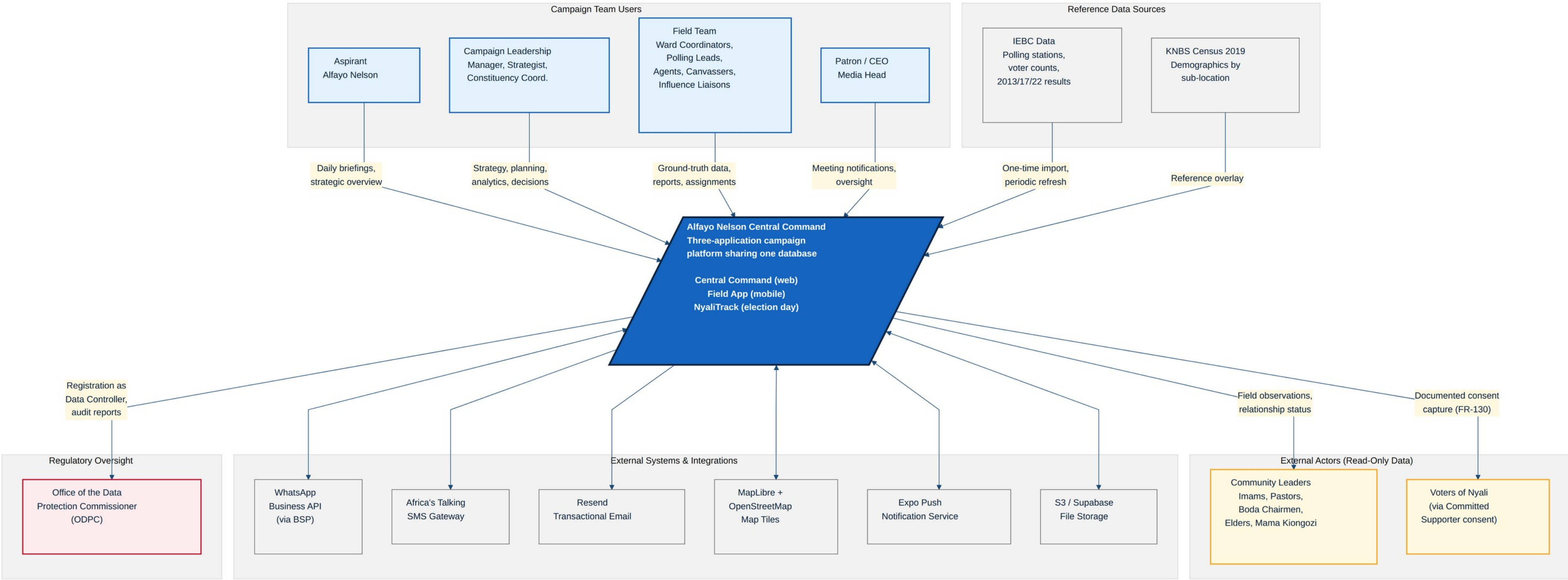
Several visual conventions are used consistently across the diagrams:

Visual Element	Meaning
Blue fill, white text	The platform itself, or a primary database / data store.
Light blue fill	A client application, user, or major actor.

Visual Element	Meaning
Yellow fill	API layer, middleware, business processes, or notes/annotations.
Light grey fill	External systems, integrations, or supporting infrastructure.
Light red fill with red border	Audit log, regulatory entity, or sensitivity callout.
Solid arrow	Direct data flow, synchronous call, or primary relationship.
Dotted arrow	Audit logging side-effect or asynchronous notification.
Cylinder shape	A data store (database table, cache, or external storage).
Circle shape	A process (in DFD) or a service component.
Rectangle shape	An external entity, user, or actor.

2. System Context Diagram

Family: Context (Level 1 abstraction)



What this diagram shows

- The platform as a single box at the centre, with the three internal applications (Central Command, Field App, NyaliTrack) noted but not decomposed.
- Four classes of campaign team users (Aspirant, Campaign Leadership, Field Team, Oversight) with one-line descriptions of how they interact with the platform.
- External actors whose data the platform reflects: community leaders and consenting voters.
- External system integrations: WhatsApp Business API, SMS gateway, email, map tiles, push notifications, file storage.
- Reference data sources read by the platform: IEBC and KNBS.

- The regulator: the Office of the Data Protection Commissioner (ODPC), with the relationship of registration and audit obligations.

What this diagram deliberately does not show

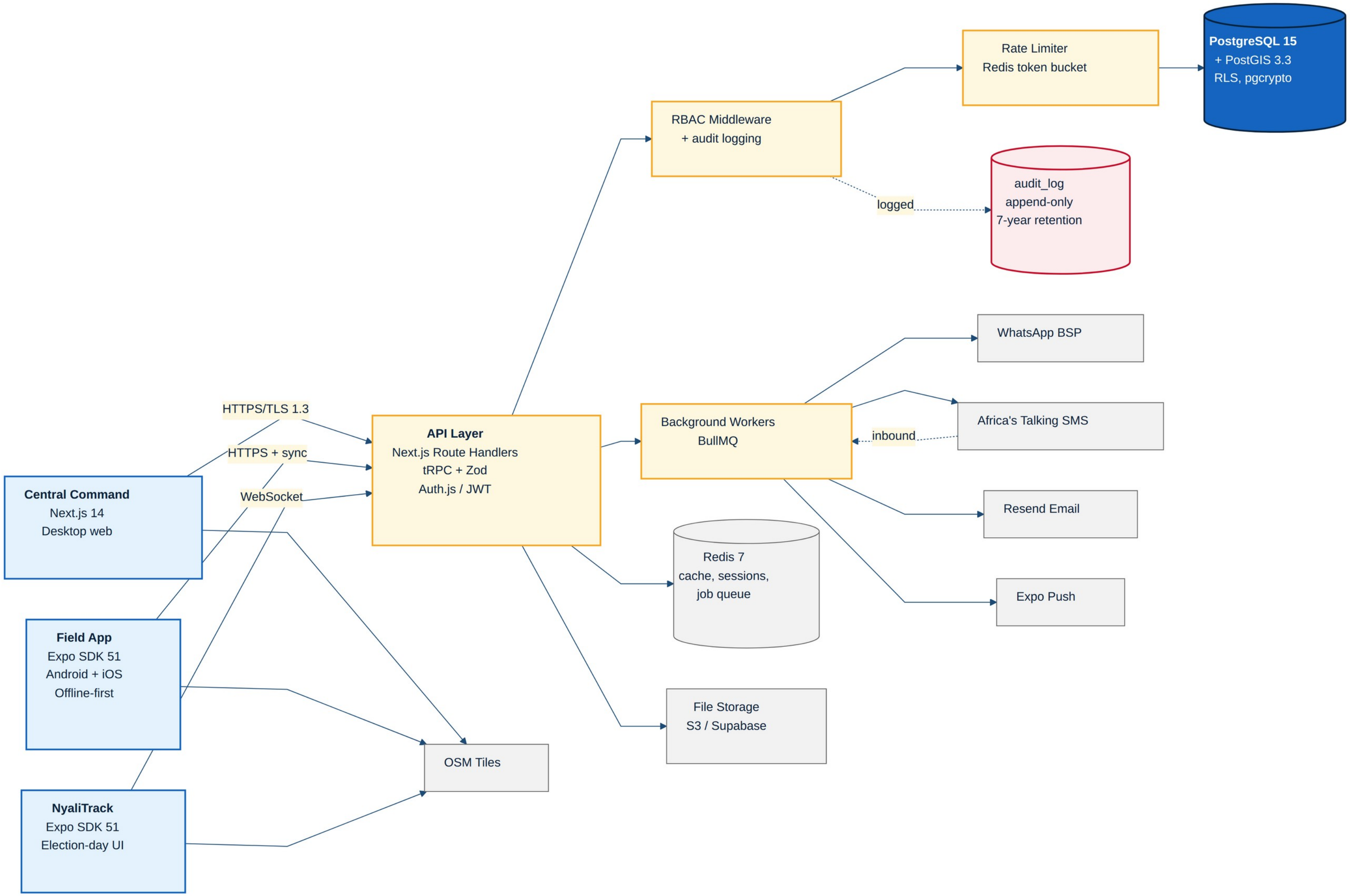
- How the platform is internally structured — see the Container Diagram (Section 3).
- The order of operations in any specific workflow — see the Sequence Diagrams (Sections 7–11).
- Specific protocols or data formats on the integration arrows — those appear in SRS Section 5.

Traces to SRS

- SRS Section 2.1 — Product Context
- SRS Section 2.2 — User Classes and Characteristics
- SRS Section 5.1 — Third-Party Integrations
- SRS Section 8 — Compliance and the ODPC relationship

3. Container / Component Diagram

Family: Container (Level 2 abstraction)


Rate Limiter
Redis token bucket
PostgreSQL 15
+ PostGIS 3.3
RLS, pgcrypto
RBAC Middleware
+ audit logging
logged
audit_log
append-only
7-year retention
WhatsApp BSP
HTTPS/TLS 1.3
Background Workers
BullMQ
Africa's Talking SMS
inbound
API Layer
Next.js Route Handlers
tRPC + Zod
Auth.js / JWT
HTTPS + sync
WebSocket
Central Command
Next.js 14
Desktop web
Resend Email
Redis 7
cache, sessions,
job queue
Expo Push
Field App
Expo SDK 51
Android + iOS
Offline-first
File Storage
S3 / Supabase
OSM Tiles
NyaliTrack
Expo SDK 51
Election-day UI

What this diagram shows

- The three client applications on the left, each labelled with its technology stack and primary platform.
- The API layer in the middle, decomposed into route handlers, RBAC middleware, the Redis rate limiter, and the BullMQ background workers.
- The data layer on the right: PostgreSQL with PostGIS as the primary store, Redis for cache and queue, the append-only audit_log table called out separately, and file storage for photos and exports.
- External services as their own group: WhatsApp BSP, Africa’s Talking SMS, Resend email, Expo Push, and OpenStreetMap tiles.
- Direction and protocol on each connection: HTTPS for client-to-API, WebSocket for NyaliTrack real-time, queue dispatch for outbound notifications, webhook for inbound SMS reports.

What this diagram deliberately does not show

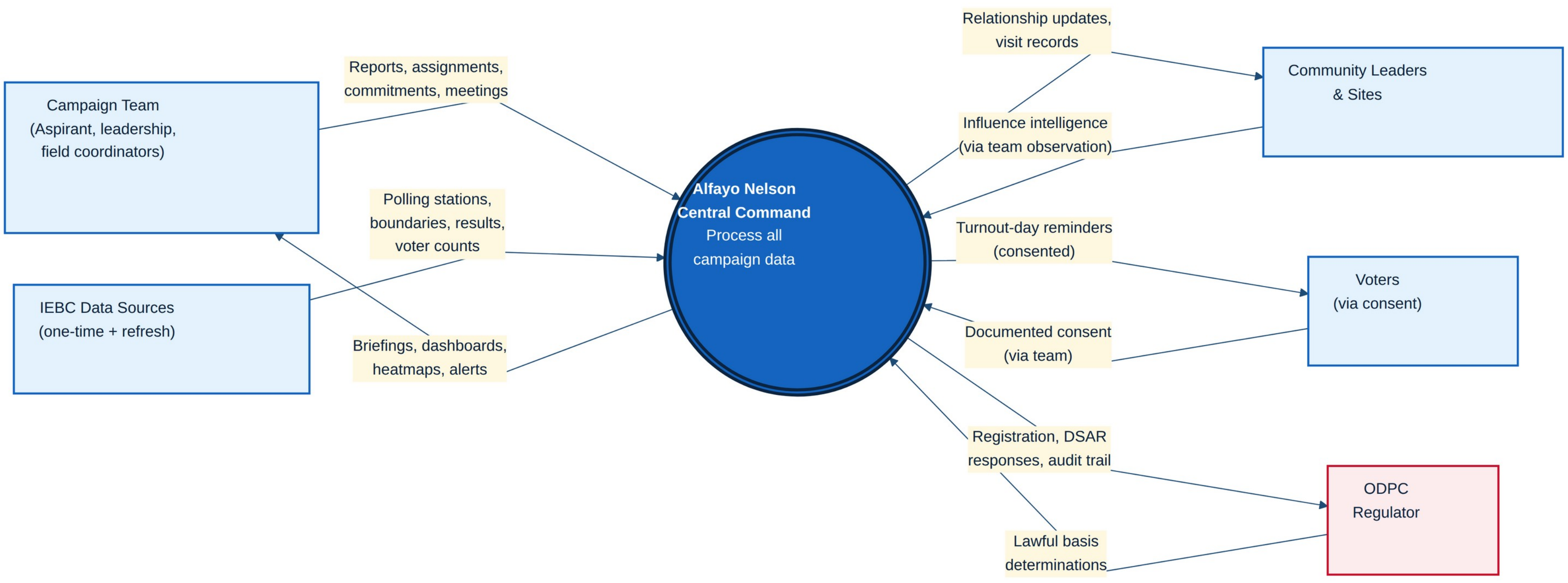
- Internal modules inside each application (controllers, services, repositories) — belongs in the Technical Specification.
- Database tables — see the ERD (Section 6).
- Deployment-specific concerns: which VPS, which region, autoscaling, load balancers — belong in the Infrastructure Document.
- Specific API endpoints — belong in the OpenAPI specification.

Traces to SRS

- SRS Section 2.3 — Operating Environment
- SRS Section 2.4 CON-003 — mandated technology stack
- SRS Section 5.2 — Technology Stack with rationale
- SRS NFR-051 — centralised logging path through the audit table

4. Data Flow Diagram — Level 0

Family: DFD (system as a single process)



What this diagram shows

- The entire platform as one process bubble in the centre.
- External entities around the platform: Campaign Team, Community Leaders, Voters (via consent), IEBC Data Sources, and ODPC Regulator.
- The major data flows in and out of the platform, labelled with the type of data crossing each boundary.
- The regulatory feedback loop with the ODPC: registration and audit responses outward, lawful basis determinations inward.

What this diagram deliberately does not show

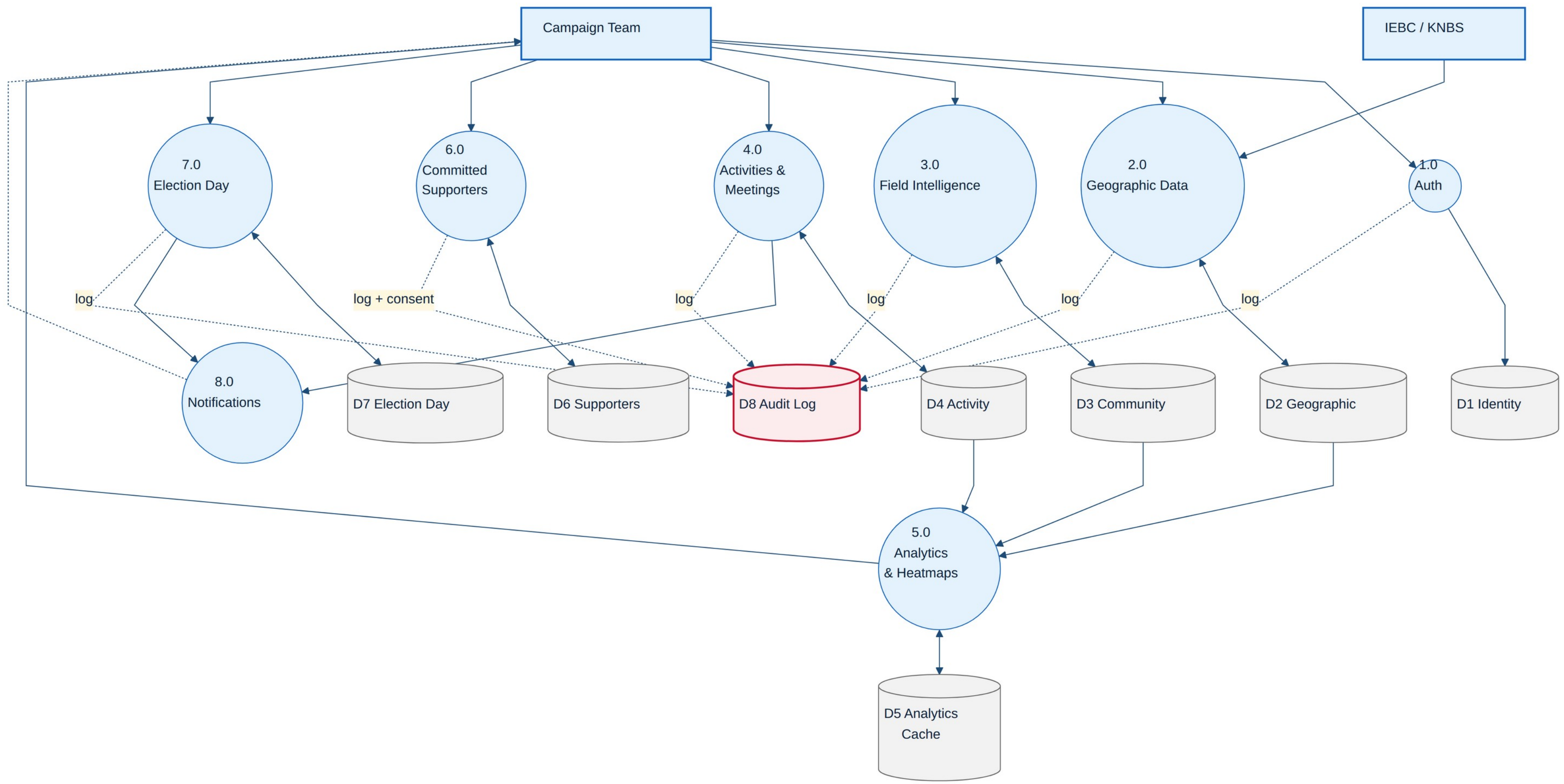
- Internal processes — see the Level 1 DFD (Section 5).
- Data stores — internal storage is invisible at Level 0 by definition.
- Timing or order of flows — DFDs are not sequence diagrams.

Traces to SRS

- SRS Section 0.2.1 — Solution Overview (the system as one bounded thing)
- SRS Section 2.1 — Product Context
- SRS Section 8.2 — Data Privacy Requirements (the ODPC relationship)

5. Data Flow Diagram — Level 1

Family: DFD (decomposed)



What this diagram shows

- Eight processes decomposed from the single Level 0 bubble: Authentication, Geographic Data, Field Intelligence, Activities and Meetings, Analytics and Heatmaps, Committed Supporters, Election Day, and Notifications.
- Eight data stores: Identity (D1), Geographic (D2), Community (D3), Activity (D4), Analytics Cache (D5), Supporters (D6), Election Day (D7), and the cross-cutting Audit Log (D8).
- The audit log is highlighted in red because it receives dotted lines from every other process — every meaningful action in the system writes to it.
- The Analytics process is downstream of multiple stores: it reads geography, community, and activity data and writes computed aggregates to the analytics cache.
- The Election Day process is structurally similar to other capture processes but operationally distinct, with its own data store and tight coupling to Notifications.

What this diagram deliberately does not show

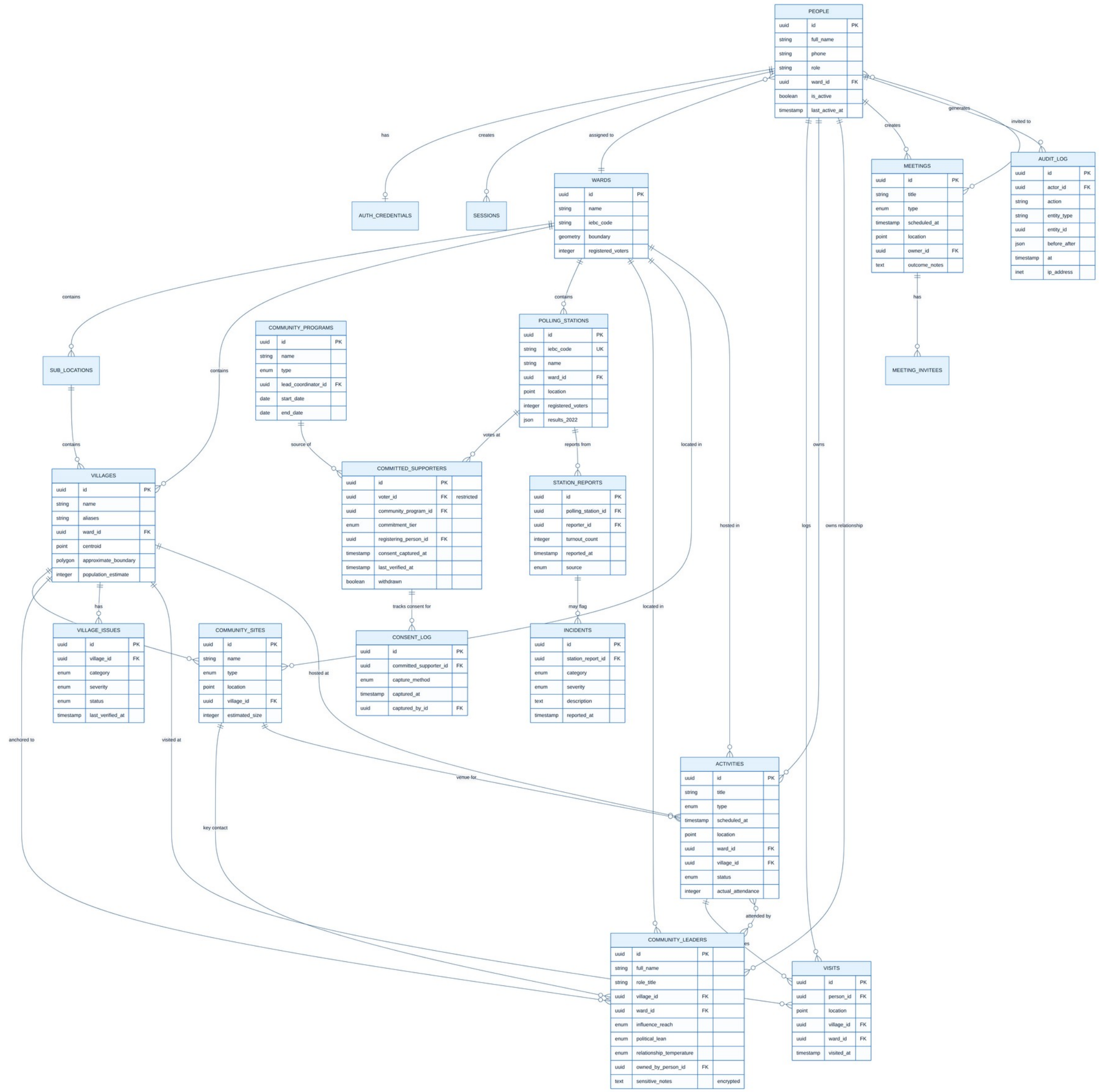
- Detailed column-level schema — see the ERD (Section 6).
- Sequencing within a process — sequence diagrams cover this.
- Permission boundaries — these are enforced inside each process via RBAC and audit logging.

Traces to SRS

- SRS Section 3 (all functional requirement domains map 1:1 to processes)
- SRS NFR-050 — append-only audit log (D8)
- SRS Section 8.2 COMP-007 — data retention by store

6. Entity Relationship Diagram

Family: ERD (logical schema)


PEOPLE
uuid id PK
string full_name
string phone
string role
uuid ward_id FK
boolean is_active
timestamp last_active_at
has
creates
assigned to
creates
generates
invited to
AUTH_CREDENTIALS
SESSIONS
WARDS
uuid id PK
string name
string iebc_code
geometry boundary
integer registered_voters
MEETINGS
uuid id PK
string title
enum type
timestamp scheduled_at
point location
uuid owner_id FK
text outcome_notes
AUDIT_LOG
uuid id PK
uuid actor_id FK
string action
string entity_type
uuid entity_id
json before_after
timestamp at
inet ip_address
has
MEETING_INVITEES
contains
contains
contains
SUB_LOCATIONS
COMMUNITY_PROGRAMS
uuid id PK
string name
enum type
uuid lead_coordinator_id FK
date start_date
date end_date
POLLING_STATIONS
uuid id PK
string iebc_code UK
string name
uuid ward_id FK
point location
integer registered_voters
json results_2022
contains
source of
votes at
reports from
located in
owns
VILLAGES
uuid id PK
string name
string aliases
uuid ward_id FK
point centroid
polygon approximate_boundary
integer population_estimate
COMMITTED_SUPPORTERS
uuid id PK
uuid voter_id FK restricted
uuid community_program_id FK
enum commitment_tier
uuid registering_person_id FK
timestamp consent_captured_at
timestamp last_verified_at
boolean withdrawn
STATION_REPORTS
uuid id PK
uuid polling_station_id FK
uuid reporter_id FK
integer turnout_count
timestamp reported_at
enum source
hosted in
logs
owns relationship
has
tracks consent for
may flag
located in
VILLAGE_ISSUES
uuid id PK
uuid village_id FK
enum category
enum severity
enum status
timestamp last_verified_at
COMMUNITY_SITES
uuid id PK
string name
enum type
point location
uuid village_id FK
integer estimated_size
hosted at
CONSENT_LOG
uuid id PK
uuid committed_supporter_id FK
enum capture_method
timestamp captured_at
uuid captured_by_id FK
INCIDENTS
uuid id PK
uuid station_report_id FK
enum category
enum severity
text description
timestamp reported_at
anchored to
visited at
ACTIVITIES
uuid id PK
string title
enum type
timestamp scheduled_at
point location
uuid ward_id FK
uuid village_id FK
enum status
integer actual_attendance
venue for
key contact
attended by
COMMUNITY_LEADERS
uuid id PK
string full_name
string role_title
uuid village_id FK
uuid ward_id FK
enum influence_reach
enum political_lean
enum relationship_temperature
uuid owned_by_person_id FK
text sensitive_notes encrypted
VISITS
uuid id PK
uuid person_id FK
point location
uuid village_id FK
uuid ward_id FK
timestamp visited_at

What this diagram shows

- Nineteen primary database entities and their relationships.
- The People entity at the top centre, anchoring authentication, ownership, and activity tracking across the platform.
- The geographic hierarchy: Wards contain Sub-Locations and Villages; Villages contain Community Sites; Wards contain Polling Stations.
- The community intelligence cluster: Community Leaders, Community Sites, and Village Issues, all anchored geographically.
- The activity cluster: Activities, Visits, and Meetings linking back to people and geography.
- The supporter cluster, restricted-access: Committed Supporters, Community Programs, and the append-only Consent Log.
- The election-day cluster: Station Reports linked to Polling Stations, with Incidents linked to Station Reports.
- The Audit Log as a cross-cutting entity referenced from every other table's activity.
- Key attributes with PK/FK/UK markers and important fields called out (encrypted fields, geographic types).

What this diagram deliberately does not show

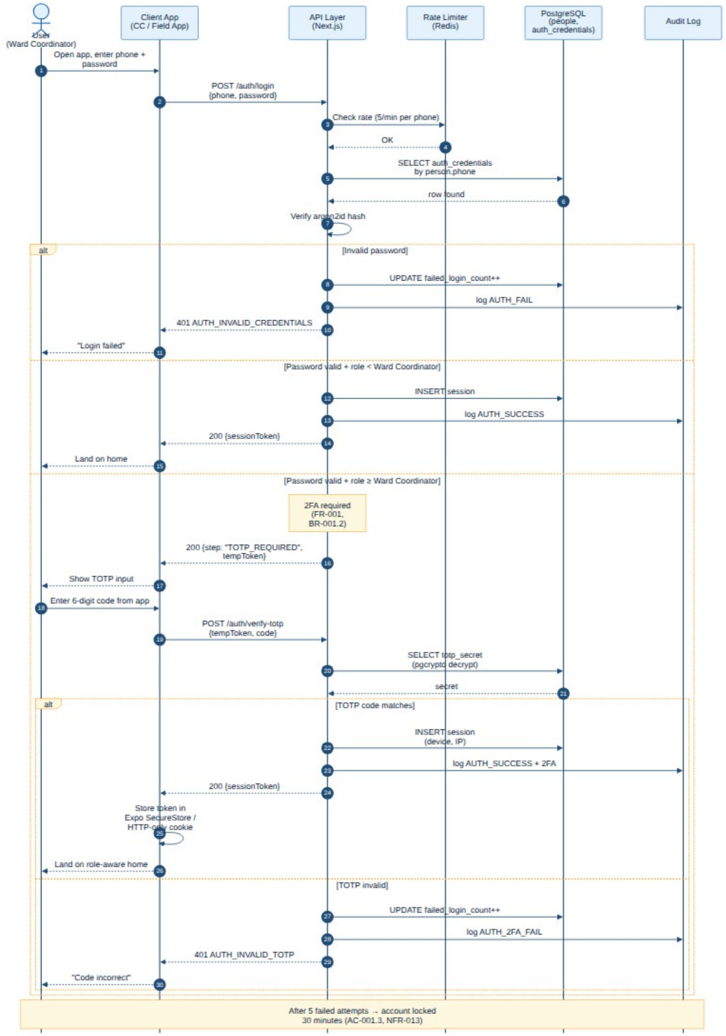
- Indexes, materialised views, and physical storage organisation — belong in the Database Design Document.
- Many-to-many junction tables not central to understanding the model (e.g. activity_photos, meeting_attachments).
- Row-level security policies — these are enforced in code/database not in the schema diagram; documented in SRS FR-002.

Traces to SRS

- SRS Section 3 — all functional requirement domains
- SRS FR-002 — row-level security policies operate over this schema
- SRS NFR-012 — column-level encryption fields shown in italic on sensitive entities

7. Sequence — Login with Two-Factor Authentication

Family: Sequence (workflow)


User
(Ward Coordinator)
Client App
(CC / Field App)
API Layer
(Next.js)
Rate Limiter
(Redis)
PostgreSQL
(people,
auth_credentials)
Audit Log
Open app, enter phone + password
POST /auth/login
{phone, password}
Check rate (5/min per phone)
OK
SELECT auth_credentials
by person.phone
row found
Verify argon2id hash
alt
[Invalid password]
UPDATE failed_login_count++
log AUTH_FAIL
401 AUTH_INVALID_CREDENTIALS
"Login failed"
[Password valid + role < Ward Coordinator]
INSERT session
log AUTH_SUCCESS
200 {sessionToken}
Land on home
[Password valid + role ≥ Ward Coordinator]
2FA required
(FR-001,
BR-001.2)
200 {step: "TOTP_REQUIRED",
tempToken}
Show TOTP input
Enter 6-digit code from app
POST /auth/verify-totp
{tempToken, code}
SELECT totp_secret
(pgcrypto decrypt)
secret
alt
[TOTP code matches]
INSERT session
(device, IP)
log AUTH_SUCCESS + 2FA
200 {sessionToken}
Store token in
Expo SecureStore /
HTTP-only cookie
Land on role-aware home
[TOTP invalid]
UPDATE failed_login_count++
log AUTH_2FA_FAIL
401 AUTH_INVALID_TOTP
"Code incorrect"
After 5 failed attempts → account locked
30 minutes (AC-001.3, NFR-013)

What this diagram shows

- The complete authentication flow from app open to landed session, covering both single-factor (canvassers, polling agents) and two-factor (ward coordinators and above) paths.
- Rate-limit checking against Redis before any password verification.
- Argon2id password verification in the API layer.
- TOTP secret retrieval with pgcrypto decryption for the 2FA path.
- Audit log entries written for every authentication event: AUTH_SUCCESS, AUTH_FAIL, AUTH_2FA_FAIL.
- The three alternative branches: invalid password, valid password without 2FA, and valid password with 2FA enrollment.
- The account lockout policy at the bottom of the diagram: 5 failed attempts triggers 30-minute lockout.

What this diagram deliberately does not show

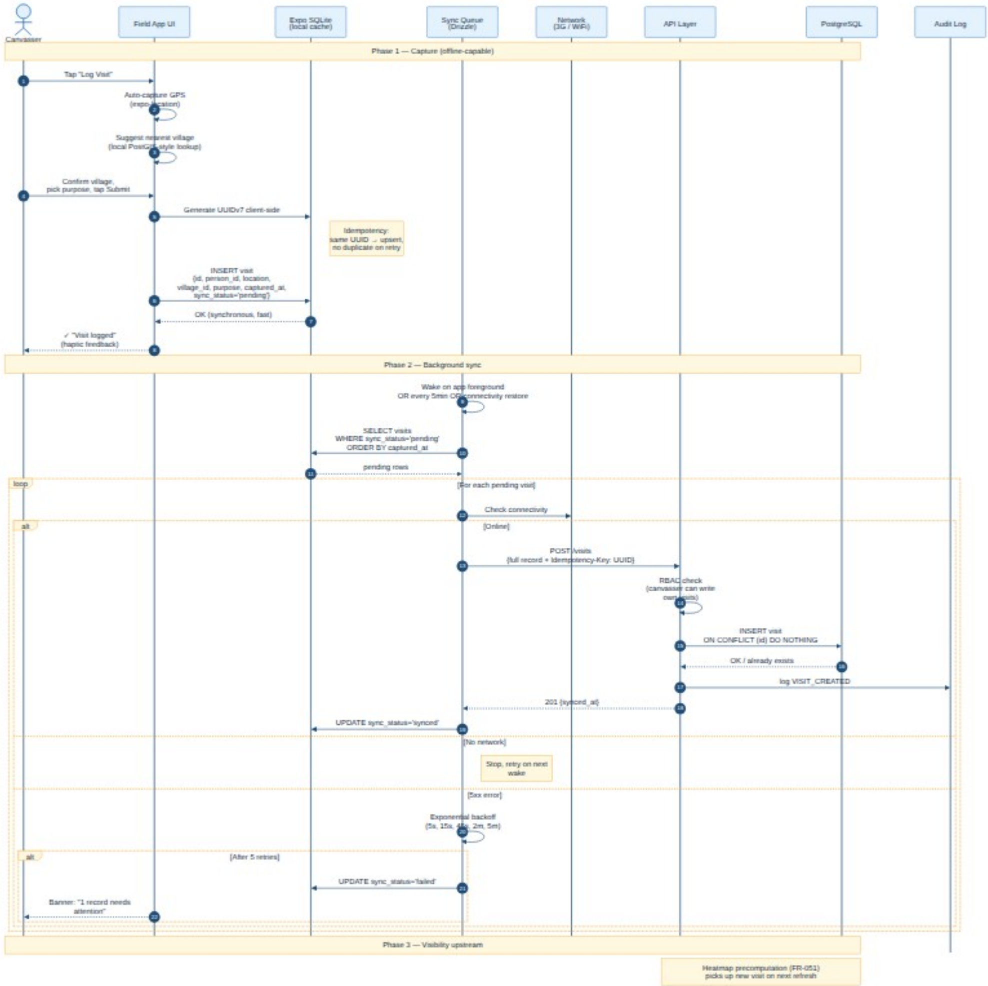
- Session refresh and token renewal flows — a separate sequence (not produced in this document).
- Password reset and recovery flows — separate sequence (not produced).
- Specific HTTP status codes for every error — see SRS FR-001 Error Handling.

Traces to SRS

- SRS FR-001 — User Authentication (including all preconditions, inputs, postconditions, and acceptance criteria)
- SRS FR-002 — Role-Based Access Control (RBAC check runs after this sequence on every subsequent request)
- SRS NFR-013 — Authentication rate limiting (the rate-limiter step)
- SRS NFR-052 — PII redaction (passwords and TOTP secrets never logged)

8. Sequence — Offline Visit Log and Sync

Family: Sequence (workflow)



Canvasser
Field App UI
Expo SQLite
(local cache)
Sync Queue
(Drizzle)
Network
(3G / WiFi)
API Layer
PostgreSQL
Audit Log
Phase 1 — Capture (offline-capable)
Tap "Log Visit"
Auto-capture GPS
Confirm village,
pick purpose, tap Submit
Generate UUIDv7 client-side
Idempotency:
same UUID → upsert,
no duplicate on retry
INSERT visit
(id, person_id, location,
village_id, purpose, captured_at,
sync_status='pending')
OK (synchronous, fast)
✓ "Visit logged"
(haptic feedback)
Phase 2 — Background sync
Wake on app foreground
OR every 5min OR connectivity restore
SELECT visits
WHERE sync_status='pending'
ORDER BY captured_at
pending rows
loop
[For each pending visit]
Check connectivity
alt
[Online]
POST /visits
(full record + Idempotency-Key: UUID)
RBAC check
(canvasser can write
INSERT visit
ON CONFLICT (id) DO NOTHING
OK / already exists
log VISIT_CREATED
201 {synced_at}
UPDATE sync_status='synced'
[No network]
Stop, retry on next
wake
[5xx error]
Exponential backoff
alt
[After 5 retries]
UPDATE sync_status='failed'
Banner: "1 record needs
attention"
Phase 3 — Visibility upstream
Heatmap precomputation (FR-051)
picks up new visit on next refresh

What this diagram shows

- The three phases of the offline-first visit logging flow: local capture, background sync, and upstream visibility.
- Phase 1: GPS auto-capture, village suggestion via local spatial lookup, three-tap submission, immediate local persistence in Expo SQLite, instant UI feedback with haptic confirmation.
- Client-side UUID generation as the idempotency mechanism: the same UUID retried multiple times produces no duplicates in the server database thanks to ON CONFLICT DO NOTHING.
- Phase 2: background sync worker wakes on app foreground, scheduled timer, or connectivity restoration; iterates over pending submissions; uses exponential backoff on retry.
- The three outcome branches per submission: online success, offline (stop and retry), persistent failure (mark for user attention).
- Phase 3: synced visits become visible upstream and feed the heatmap precomputation pipeline (linking to Sequence 11).

What this diagram deliberately does not show

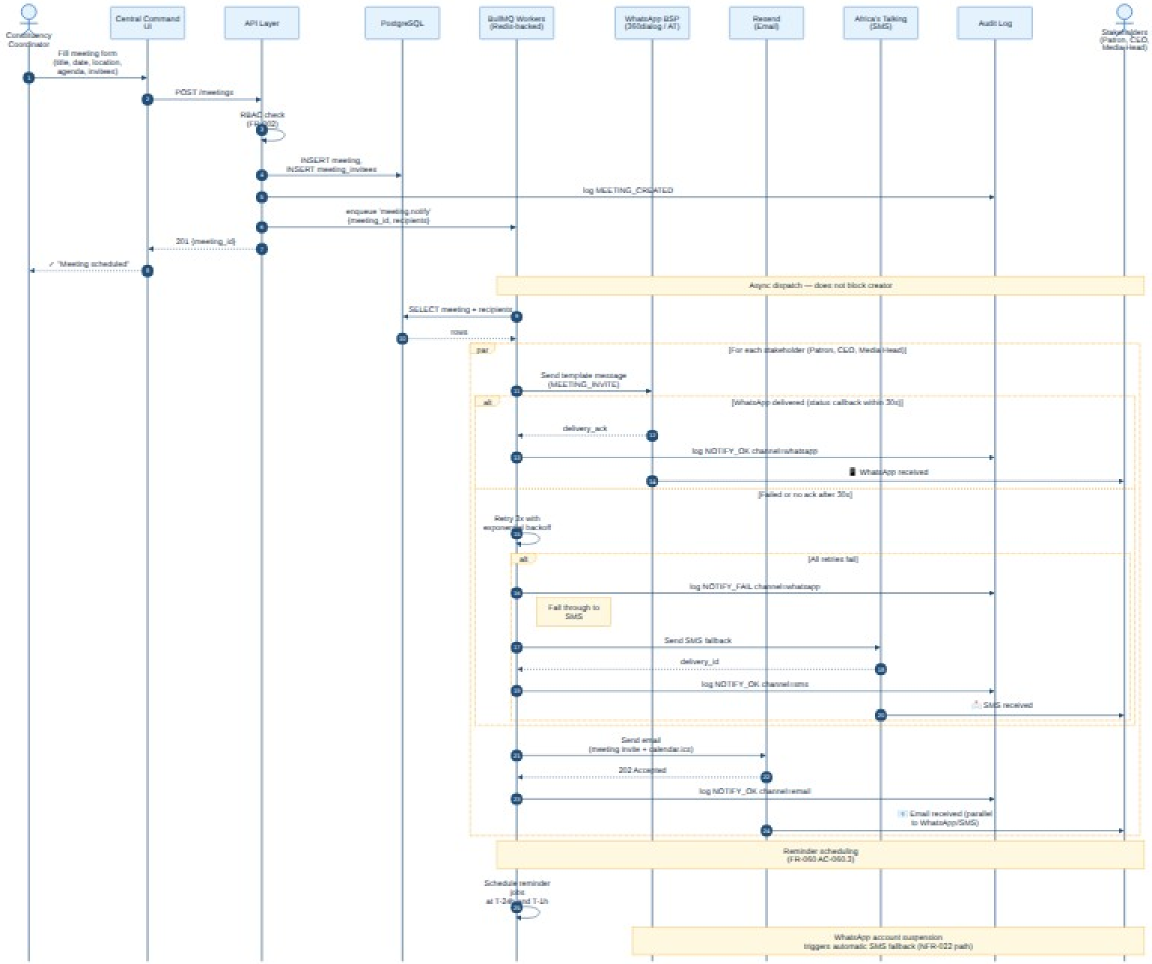
- Photo upload sub-flow within a visit — photos are queued separately to avoid blocking small text submissions.
- Conflict resolution for mutable entities (only relevant for entities the user can edit; visits are append-only).
- Auth token refresh during long offline periods — handled by the auth subsystem.

Traces to SRS

- SRS FR-050 — Three-Tap Visit Logging (including AC-050.3 offline capability and AC-050.4 sync timing)
- SRS FR-100 — Offline-First Operation (24-hour offline floor, idempotency, exponential backoff)
- SRS NFR-002 — Field App load time on 3G
- SRS CON-008 — offline operation constraint

9. Sequence — Create Meeting with WhatsApp, Email, and SMS Fallback

Family: Sequence (workflow)



Constituency Coordinator
Central Command UI
API Layer
PostgreSQL
BullMQ Workers (Redis-backed)
WhatsApp BSP (360dialog / AT)
Resend (Email)
Africa's Talking (SMS)
Audit Log
Stakeholders (Patron, CEO, Media Head)
Fill meeting form (title, date, location, agenda, invitees)
POST /meetings
RBAC check
INSERT meeting, INSERT meeting_invitees
log MEETING_CREATED
enqueue 'meeting.notify' {meeting_id, recipients}
201 {meeting_id}
✓ 'Meeting scheduled'
Async dispatch — does not block creator
SELECT meeting + recipients
rows
par
[For each stakeholder (Patron, CEO, Media Head)]
Send template message (MEETING_INVITE)
alt
[WhatsApp delivered (status callback within 30s)]
delivery_ack
log NOTIFY_OK channel=whatsapp
WhatsApp received
[Failed or no ack after 30s]
Retry 3x with exponential backoff
alt
[All retries fail]
log NOTIFY_FAIL channel=whatsapp
Fall through to SMS
Send SMS fallback
delivery_id
log NOTIFY_OK channel=sms
SMS received
Send email (meeting invite + calendar.ics)
202 Accepted
log NOTIFY_OK channel=email
Email received (parallel to WhatsApp/SMS)
Reminder scheduling (FR-060 AC-060.3)
Schedule reminder jobs at T-24h and T-1h
WhatsApp account suspension triggers automatic SMS fallback (NFR-022 path)

What this diagram shows

- Meeting creation through Central Command, including database insert and immediate audit log entry.
- The non-blocking dispatch pattern: the user receives confirmation before notifications are sent, because notifications run on the BullMQ worker queue.
- Parallel dispatch to multiple stakeholders (patron, CEO, media head) per the configured notification list.
- The WhatsApp-first, SMS-fallback pattern: 3 retries with exponential backoff, then automatic SMS fallback on persistent failure.
- Email dispatched in parallel to WhatsApp/SMS (not as fallback) because email and WhatsApp serve different attention patterns.
- Reminder job scheduling at T-24 hours and T-1 hour.
- Audit log entries for every successful and every failed notification attempt.

What this diagram deliberately does not show

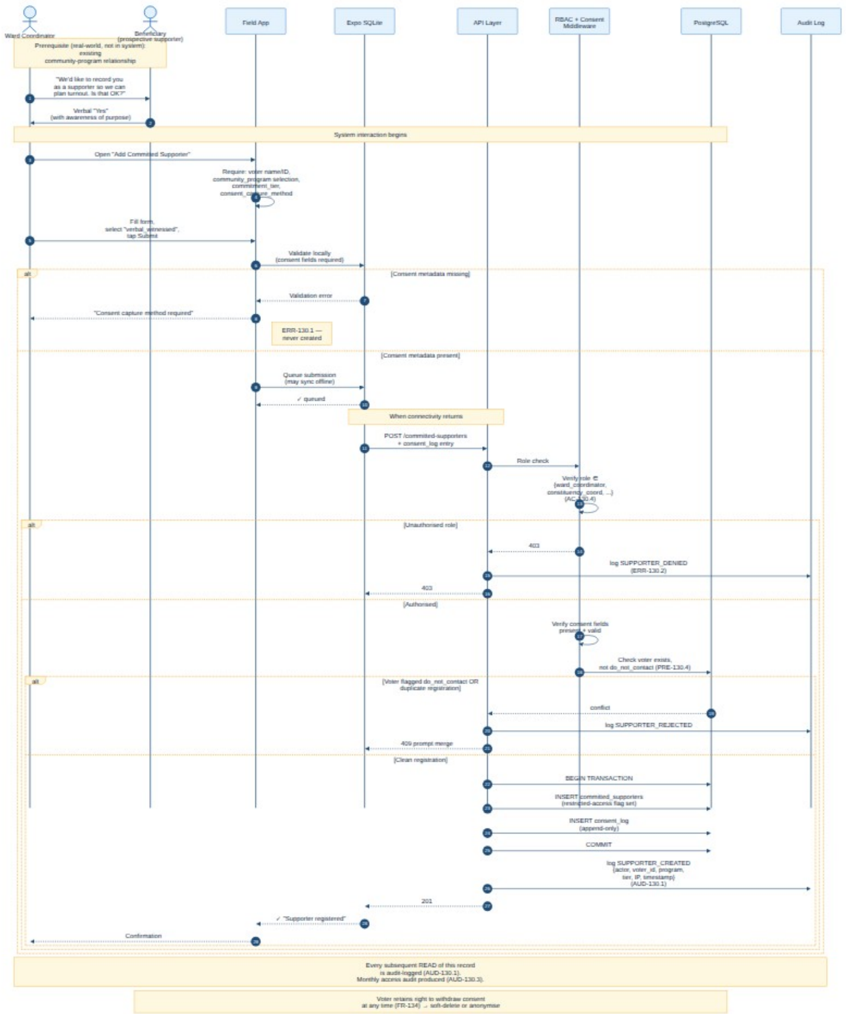
- Meeting modification and cancellation flows — similar shape but emit different notification templates.
- Calendar integration (iCal) generation — handled within the email dispatch step.
- Recipient consent handling — covered separately in SRS COMP-004.

Traces to SRS

- SRS FR-060 — Meeting Creation with Automated Notifications
- SRS FR-060 AC-060.3 — reminder scheduling
- SRS FR-060 AC-060.4 and ERR-060.1 — WhatsApp—to—SMS fallback
- SRS NFR-022 — reliability and fallback path on integration failure
- SRS COMP-005 — user consent (recipients must be on the consent list)

10. Sequence — Register Committed Supporter

Family: Sequence (workflow — sensitive)



Ward Coordinator
Beneficiary (prospective supporter)
Field App
Expo SQLite
API Layer
RBAC + Consent Middleware
PostgreSQL
Audit Log
Prerequisite (real-world, not in system): existing community-program relationship
"We'd like to record you as a supporter so we can plan turnout. Is that OK?"
Verbal "Yes" (with awareness of purpose)
System interaction begins
Open "Add Committed Supporter"
Require: voter name/ID, community_program selection, commitment_tier, consent_capture_method
Fill form, select "verbal_witnessed", tap Submit
Validate locally (consent fields required)
alt
[Consent metadata missing]
Validation error
"Consent capture method required"
ERR-130.1 — never created
[Consent metadata present]
Queue submission (may sync offline)
✓ queued
When connectivity returns
POST /committed-supporters + consent_log entry
Role check
Verify role ∈ {ward_coordinator, constituency_coord, ...} (AC-130.4)
alt
[Unauthorised role]
403
log SUPPORTER_DENIED (ERR-130.2)
403
[Authorised]
Verify consent fields present + valid
Check voter exists, not do_not_contact (PRE-130.4)
alt
[Voter flagged do_not_contact OR duplicate registration]
conflict
log SUPPORTER_REJECTED
409 prompt merge
[Clean registration]
BEGIN TRANSACTION
INSERT committed_supporters (restricted-access flag set)
INSERT consent_log (append-only)
COMMIT
log SUPPORTER_CREATED {actor, voter_id, program, tier, IP, timestamp} (AUD-130.1)
201
✓ "Supporter registered"
Confirmation
Every subsequent READ of this record is audit-logged (AUD-130.1). Monthly access audit produced (AUD-130.3).
Voter retains right to withdraw consent at any time (FR-134) → soft-delete or anonymise

What this diagram shows

- The real-world consent capture step that must precede any system interaction. The voter (beneficiary) must give explicit awareness-of-purpose consent.
- Local validation on the Field App: the system refuses to even queue a submission without consent metadata.
- The role-check enforcement: only specific named roles can create supporter records (ward_coordinator and above per AC-130.4).
- The transactional database write: committed_supporters and consent_log inserts happen in a single transaction; failure rolls back both.
- The 404 (not 403) response on unauthorised reads: this avoids confirming the existence of a sensitive record to a probing attacker.
- The append-only audit log entries that fire on every create, read, denial, and rejection — supporting the monthly access audit (AUD-130.3).
- The withdrawal-of-consent path noted at the bottom: voters retain the right to soft-delete or anonymise their record at any time.

What this diagram deliberately does not show

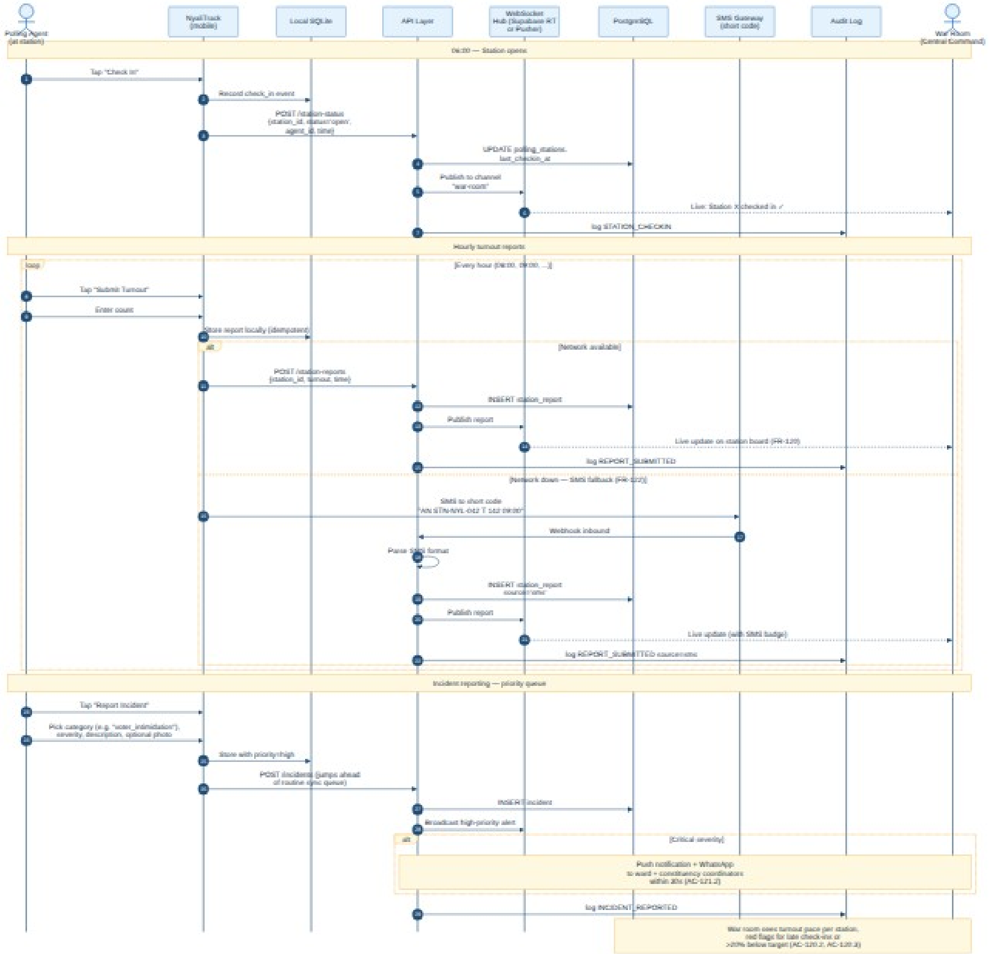
- The merge-duplicates UI flow when a duplicate is detected — belongs in the UX Specification.
- The withdrawal procedure detail — covered by FR-134 and not detailed in this sequence.
- The monthly access audit report generation — separate scheduled job not shown here.

Traces to SRS

- SRS FR-130 — Committed Supporter Registration (all preconditions, inputs, postconditions, acceptance criteria, business rules)
- SRS FR-131 — Community Program Registry (the program reference required)
- SRS FR-134 — Consent Withdrawal and Right to Erasure
- SRS COMP-010 — Committed Supporter Data Safeguards
- SRS Risk Matrix — "Leak of Committed Supporter list" mitigation

11. Sequence — Election-Day Polling Station Report

Family: Sequence (workflow — high-load)



Polling Agent (at station)
NyatiTrack (mobile)
Local SQLite
API Layer
WebSocket Hub (Supabase RT or Pusher)
PostgreSQL
SMS Gateway (short code)
Audit Log
War Room (Central Command)
06:00 — Station opens
Tap "Check In"
Record check_in event
POST /station-status (station_id, status=open, agent_id, time)
UPDATE polling_stations. last_checkin_at
Publish to channel "war-room"
Live: Station X checked in ✓
log STATION_CHECKIN
Hourly turnout reports
loop
[Every hour (08:00, 09:00, ...)]
Tap "Submit Turnout"
Enter count
Store report locally (idempotent)
alt
[Network available]
POST /station-reports (station_id, turnout, time)
INSERT station_report
Publish report
Live update on station board (FR-120)
log REPORT_SUBMITTED
[Network down — SMS fallback (FR-122)]
SMS to short code "AN STN-NYL-042 T 142 09:00"
Webhook inbound
Parse SMS format
INSERT station_report source='sms'
Publish report
Live update (with SMS badge)
log REPORT_SUBMITTED source=sms
Incident reporting — priority queue
Tap "Report Incident"
Pick category (e.g. "voter_intimidation"), severity, description, optional photo
Store with priority=high
POST /incidents (jumps ahead of routine sync queue)
INSERT incident
Broadcast high-priority alert
alt
[Critical severity]
Push notification + WhatsApp to ward + constituency coordinators within 30s (AC-121.2)
log INCIDENT_REPORTED
War room sees turnout pace per station, red flags for late check-ins or >20% below target (AC-120.2, AC-120.3)

What this diagram shows

- The full election-day reporting cycle for a polling station: check-in at 06:00, hourly turnout reports through the day, and ad-hoc incident reports.
- The WebSocket real-time path: every report and incident is broadcast to the war room dashboard within seconds.
- The SMS fallback path: when data network is unavailable, an agent can SMS a structured short-code message that the system parses and stores identically to an in-app submission.
- The priority queue for incident reports: incidents jump ahead of routine reports in the sync queue.
- The critical-severity branch: critical incidents trigger push + WhatsApp notifications to the ward and constituency coordinators within 30 seconds.
- The war-room monitoring view that surfaces late check-ins (red flag) and below-target turnout pace (amber flag).

What this diagram deliberately does not show

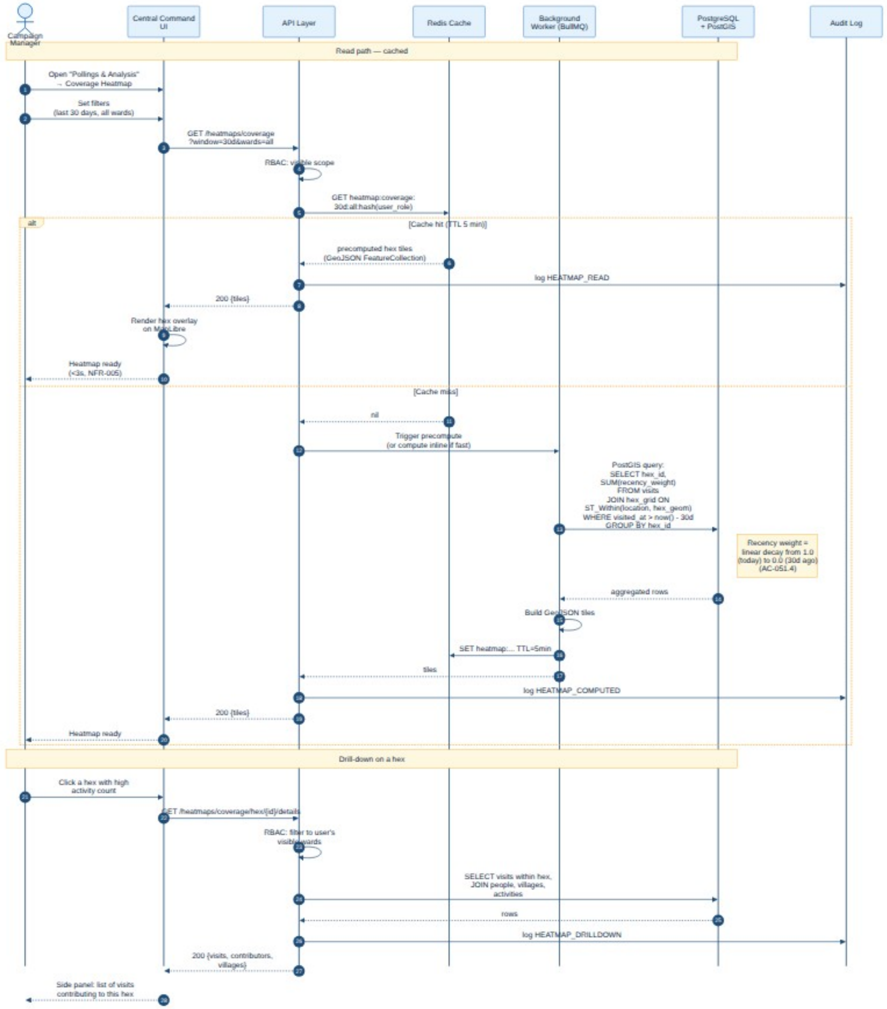
- The full incident workflow including verification, escalation to legal counsel, and IEBC reporting — these are operational procedures outside the system.
- The end-of-day results submission flow — separate sequence to be detailed in SRS-ALFAYO-002.
- Specific SMS short-code parsing rules — belong in the operations runbook.

Traces to SRS

- SRS FR-120 — Polling Station Status Board
- SRS FR-121 — Structured Incident Reporting (with AC-121.2 critical-severity notification path)
- SRS FR-122 — SMS Fallback for D-Day
- SRS NFR-021 — Election-Day Uptime (99.95% over 72-hour window)
- SRS NFR-007 — Concurrent User capacity 500 on D-Day

12. Sequence — Generate Coverage Heatmap

Family: Sequence (workflow — analytics)


Campaign Manager
Central Command UI
API Layer
Redis Cache
Background Worker (BullMQ)
PostgreSQL + PostGIS
Audit Log
Read path — cached
Open "Pollings & Analysis" → Coverage Heatmap
Set filters (last 30 days, all wards)
GET /heatmaps/coverage ?window=30d&wards=all
RBAC: visible scope
GET heatmap:coverage: 30d:all:hash(user_role)
alt
[Cache hit (TTL 5 min)]
precomputed hex tiles (GeoJSON FeatureCollection)
log HEATMAP_READ
200 {tiles}
Render hex overlay on MapLibre
Heatmap ready (<3s, NFR-005)
[Cache miss]
nil
Trigger precompute (or compute inline if fast)
PostGIS query: SELECT hex_id, SUM(recency_weight) FROM visits JOIN hex_grid ON ST_Within(location, hex_geom) WHERE visited_at > now() - 30d GROUP BY hex_id
Recency weight = linear decay from 1.0 (today) to 0.0 (30d ago) (AC-051.4)
aggregated rows
Build GeoJSON tiles
SET heatmap:... TTL=5min
tiles
log HEATMAP_COMPUTED
200 {tiles}
Heatmap ready
Drill-down on a hex
Click a hex with high activity count
GET /heatmaps/coverage/hex/{id}/details
RBAC: filter to user's visible wards
SELECT visits within hex, JOIN people, villages, activities
rows
log HEATMAP_DRILLDOWN
200 {visits, contributors, villages}
Side panel: list of visits contributing to this hex

What this diagram shows

- The cache-first read path: Redis is checked before any database work; cache hits return precomputed hex tiles immediately.
- The cache miss path: a background worker performs the PostGIS aggregation query (visits grouped by hex with recency weighting), builds the GeoJSON tile set, writes to cache with a 5-minute TTL, and returns to the caller.
- The recency weighting formula referenced in AC-051.4: linear decay from 1.0 today to 0.0 at the window’s end.
- The drill-down path: clicking a hex triggers a follow-up query that returns the specific visits and contributors that produced the hex’s aggregate.
- Audit log entries on every read of analytics data, supporting access pattern monitoring.

What this diagram deliberately does not show

- The cache invalidation strategy on new visit insertion — handled by the background worker queue not shown here.
- The two related heatmaps (Influence Density, Persuasion Opportunity) which follow the same pattern with different SQL.
- The export-to-PNG path for sharing visualisations — separate flow.

Traces to SRS

- SRS FR-051 — Coverage Heatmap (Heatmap A) with all acceptance criteria
- SRS FR-052, FR-053 — Influence Density and Persuasion Opportunity heatmaps (same pattern)
- SRS NFR-005 — Heatmap render time ≤3 seconds
- SRS NFR-001 — Page load ≤2 seconds (cache hit case)

Document Control

Revision History

Version	Date	Author	Summary of Changes
v0.1	26 May 2026	Developers Mania Technical Lead	Initial draft. Eleven architecture diagrams produced (1 Context, 1 Container, 2 DFD, 1 ERD, 6 Sequence). Companion to SRS-ALFAYO-001 v0.2.
v0.2	[Pending]	[Author]	[Incorporate peer review feedback]
v1.0	[Pending]	[Author]	[First signed version after Technical Lead, Account Lead, Client, and Founding Partner sign-off]

Approval Sign-Off

Signed alongside SRS-ALFAYO-001. Approvers should sign both documents together; the architecture diagrams are not an independent contract but a visualisation of the SRS.

Role	Name	Signature	Date
Technical Lead	[To be assigned]		
Account Lead	[To be assigned]		
Client Representative (Campaign Manager)	[To be assigned]		
Aspirant	Alfayo Nelson		
Founding Partner	[To be assigned]		